

SQA

0. Introduction

1. SDLC

1. requirements analysis
2. design
3. development
4. testing
5. maintenance

2. STLC

1. requirement analysis
2. test planning
3. test case development
4. environment setup
5. test execution
6. test cycle closure

3. **Software quality** = the quality that ensures customer satisfaction by offering all the customer deliverables on performance, standards and ease of operations.
4. **Quality assurance** = system of activities designed to ensure production that meets pre-established requirements and standards.
5. **Testing** = an activity to check whether the actual results match the expected results and to ensure that the software system is defect free.

6. Why does SW have bugs

1. miscommunication or no communication
2. exponential growth in system complexity
3. programming errors
4. dependencies among code modules/services/systems
5. changing requirements
6. time pressure
7. poorly designed/documented code
8. sw dev tools

7. Problems and solutions

1. poor requirements - solid requirements/user stories
2. unrealistic schedule - realistic schedules

3. Inadequate testing - adequate testing
 4. misunderstandings about dependencies or features - stick to the initial requirements where feasible
 5. miscommunication - communication
-
-

1. Software Quality

1. **Software** = computer programs, procedures, and possible associated documentation and data pertaining to the operation of a computer system.
2. Software defects

1. errors

1. sections of the code
2. grammatical or logical or other mistakes
3. by dev, systems analyst or any other member
4. e.g:

```
def add(a: int, b: int) -> int:  
    return a * b
```

5. may or may not be detected before release
6. not all errors become faults

2. faults

1. result of an error
2. causes incorrect functioning
3. during the execution of the program
4. not all faults are detected

3. failures

1. faults become failures
2. only when the particular function is activated
3. detectable problem

3. devs are interested in - errors and faults

1. users - failures

4. Reasons why SQA fails

1. faulty requirements definition

2. client-dev communication failures
 3. deliberate deviations from software requirements
 1. reusing old modules
 2. time/budget pressures
 4. logical design errors
 5. coding errors
 6. non-compliance with documentation and coding instructions
 1. non-complying member
 7. shortcomings of the testing process
 1. incomplete test plans
 2. incomplete correction of detected errors
 8. procedure errors
 9. documentation errors
 1. omission of functions
5. **Software quality** = conformance to explicitly stated functional and performance requirements and standards.
6. **Software quality control** = a set of activities designed to evaluate the quality
7. **Software quality assurance** = activities to minimize the cost of guaranteeing quality by a variety of activities performed throughout the development process
8. Objectives of SQA
1. in software development
 1. conforming to functional technical reqs, managerial scheduling and budgetary reqs
 2. improvement and efficiency of the sw dev process
 2. software maintenance
 1. conforming to functional technical reqs, managerial scheduling and budgetary reqs
-
-

2. Software Quality Challenge

1. software and software QA is unique from other products and their QA process
2. characteristics
 1. product complexity - operational modes of the prod
 2. product visibility

3. limited opportunities to detect bugs/defects
 1. product development phase
 2. product productional planning phase
 3. manufacturing phase
 3. Characteristics of SQA environment
 1. subject to customer-supplier relationship
 2. contractual conditions
 3. teamwork
 4. maintenance of the SW for years
-
-

3. SQ Factors

1. SQ factors
 1. directly measurable
 2. indirectly measurable
2. SQA models that categorize the sw reqs into factors
 1. McCall's factor model - 11 factors
 2. The Evans and Marciniak factor model - 12
 1. design
 2. performance
 3. adaptation
 - verifiability
 - expandability
 3. The Deutsch and Willis factor model - 15
 1. functional
 2. performance
 3. change
 4. management
 - verifiability
 - expandability
 - safety
 - manageability
 - survivability
 - Testability removed

McCall's Classic Factor Model

1. factors
 1. product revision
 1. maintainability
 2. flexibility
 3. testability
 2. product transition
 1. portability
 2. reusability
 3. interoperability
 3. product operation
 1. correctness
 2. reliability
 3. efficiency
 4. integrity
 5. usability
-
-

4. Components of SQA

1. classes of SQA system components
 1. pre project components
 2. project life cycle activities components
 3. infrastructure components
 4. SQ management
 5. standardization, certification and SQA assessments
 6. organizing for SQA - the human components

Pre project component

1. goal
 2. proper definitions
 3. proper planning
2. components
 1. Contract review

2. development and quality plans

Project life cycle components

1. stages
 1. development life cycle stage
 2. operation-maintenance stage
2. components
 1. reviews
 1. formal design review
 2. peer reviews
 1. inspections
 2. walkthroughs
 2. expert opinions
 3. software testing
 1. modules
 2. integration
 3. whole system
 4. software maintenance
 1. corrective
 2. adaptive
 3. preventive
 4. perfective
 5. assurance of the quality of any external participants
 1. programmers
 2. testers

-
1. infrastructure SQA components
 1. work instructions
 2. staff training, certification
 3. documentation
 4. configuration management
 2. management SQA components
 1. project progress
 2. software quality metrics
 3. software quality costs

3. SQA standards, system cert and assessment
 1. quality management standards
 1. SEI CMM assessment standard
 2. ISO9001 and ISO 9000-3 standards
 2. project process standards
 1. IEEE 1012 standards
 2. ISO/IEC 12207 standard
 4. Organizing for SQA - the human components
 1. managers
 2. testing personnel
 3. SQA committee and practitioners
-
-

5. Types of testing

1. Benefits of testing
 1. cost effective
 2. customer satisfaction
2. Manual testing
 1. test plan, test cases, test scenario
 2. Manual testing steps
3. Automated testing
 1. What to automate
4. Functional testing
 1. tests functional requirements
 2. many types
5. Non-functional testing
 1. stability, usability
 2. many types
6. Functional testing types
 1. Unit testing - single module's IO
 2. Integration test - joining more than one module
 3. Regression testing - testing related modules after one module is changed
 1. all modules
 2. selected modules

3. prioritized modules
4. smoke testing / build verification testing / confidence testing - testing to decide whether build can be passed on to the testers
 1. goal is stability
5. sanity testing - testing the updated module only
 1. goal is rationality
6. System testing
 1. overall system after all above tests are completed
7. acceptance testing
 1. kind of black box
 2. types
 1. user acceptance testing
 2. business acceptance testing
 3. alpha testing - within dev teams
 4. beta testing
7. Non-functional testing
 1. performance testing
 1. speed
 2. scalability
 3. stability
 2. Load testing
 1. max ops capacity
 2. hw/infra capability/suitability
 3. concurrent user count
 3. stress testing
 1. give more than expected load
 2. find the limits of the system
 3. types
 1. application stress testing
 2. transaction stress testing
 3. systemic stress testing
 4. exploratory stress testing - giving inputs not possible in real world
 5. distributed stress testing - distributed systems and distributed stress
 4. endurance testing
 1. like load testing but over an extended period of time
 5. spike testing
 1. increasing load in intervals

6. volume testing/flood testing
 1. populating DB with high volume data
 7. scalability testing
 1. when no. of requests is scaled up or down
-
-

6. Software Testing Strategies

1. SQA tools
 1. testing
 1. big bang
 2. incremental
 2. code inspection
 3. code walkthrough
 1. nonformal
 2. by authors
 3. unplanned and no fixed responsibilities
2. SQA testing strategies
 1. white box testing
 2. black box testing

White box testing

1. methods
 1. data processing and calculation correctness tests
 1. path coverage
 2. line coverage
 2. software qualification tests
 3. maintainability tests
 4. reusability tests

Data processing and calculation correctness tests

1. considering each case/scenario
2. path coverage
 1. every computational case or path must be examined

3. line coverage

1. every line of code must be executed during testing
2. Line coverage equation
TBD
3. can find
 1. unused statements
 2. dead code
 3. unused branches
4. cyclomatic complexity metrics

Software qualification testing

1. whether the software qualifies according to the chosen standard

Maintainability tests

1. for deployed software
2. types
 1. corrective
 2. perfective
 3. adaptive
 4. preventive

Black box testing

1. functionality testing
2. entirely based on sw reqs and specs
3. types
 1. equivalence classes for output correctness tests
 2. decision table
 3. revision factor testing
 1. maintainability
 2. flexibility
 3. testability
 4. transition factor testing
 1. portability
 2. reusability
 3. interoperability

Testing Methodologies

1. Big bang
 1. tests the system as a whole
 2. for smaller projects
 2. incremental
 1. tests modules
 2. for bigger projects
 3. types
 1. bottom up
 2. top down
 4. stubs and drivers
 1. hardcoding an undeveloped module's functionality for testing
-
-

7. Software Testing - Implementation

1. Test effectiveness
 1. reduction of the percentage of errors
 2. formula
$$\text{test effectiveness} = \frac{\text{no. of valid defects fixed}}{\text{total no. of defects injected} + \text{total no. of defects escaped}} \%$$
2. Test efficiency
 1. performance of the tests with fewer resources
 2. formula
$$\text{test efficiency} = \frac{\text{no. of alpha testing in-house defects}}{\text{no. of alpha testing defects} + \text{no. of beta testing defects}}$$

Testing process

1. testing process
 1. determining test methods
 2. planning unit, integration tests
 3. planning system test
 4. designing tests
 5. implementation
2. test methodology

1. SQ standard
2. strategy
 1. big bang or incremental?
 2. which parts to white/black box?
 3. extent of automated testing
3. test planning
 1. what to test
 2. synthetic or real-life test cases
 3. who tests
 1. same/another dev team
 2. outsourcing or independent testing team
 4. when to stop testing
 1. perfection approach
 2. mathematical model
 3. error seeding method
 4. dual independent teams route
 5. until resources run out
 5. documentation
 1. plan is documented in **Software Test Plan** document
 2. STP includes
 1. scope of the tests
 2. testing env
 3. test details
 4. test schedule including duration estimates
 4. test designing - based on STP
 1. software test procedure doc - detailed design and procedure for each test
 2. test case file - test case db/file
 3. or both of the above in Software Test Description / **STD**
 5. test implementation
 1. series of tests
 2. correction of detected defects
 3. retest (regression)
 6. documentation
 1. result of the tests and retests -> Software Test Report doc
 2. summary of set of planned tests -> Test Summary Report doc

Test Case

1. design
 1. inputs
 2. conditions
 3. expected results
2. sources
 1. real-life
 2. synthetic
3. stratified sampling - for getting test cases for every scenario

Automated testing

1. process
 1. test planning
 2. test design
 3. test case preparation
 4. test performance
 5. test log and report prep
 6. regression tests
 7. final test log, report and comparison report prep
 2. types
 1. code audit
 - 1.
 2. coverage monitoring
 3. functional tests
 4. load tests
 5. test management
 3. test management
 4. alpha and beta site testing programs
-
-

Session 4 - Emerging Trends

1. emerging trends
 1. test automation
 2. agile, TDD/BDD and DevOps
 1. test driven development

2. behavior driven development
 3. AI for testing
 4. sw testing Cybersec
 5. Data Science Testing
 6. Blockchain testing
 7. low code / no code automation
2. Challenges of testing AI systems
 1. non-deterministic
 2. probabilistic
 3. non-testable
 4. oracle-problem
 1. there isn't a defined correct answer, especially in Gen AI
3. data science lifecycle
 1. data ingestion
 2. data storage and data processing
 3. data analysis
 4. communicate
 - 4.